

Assignment 1

```
## [1] "Sebastian Hyland (39878921)"
```

Dataset overview

Origin: part of the data published by Blackmore et al. (2017)

Study: The effect of upper-respiratory infection on transcriptomic changes in the CNS

File: CSV format:

Column	Description
gene	The name of the gene that was measured
sample	The name of the sample the gene expression was measured in
expression	The value of the gene expression
organism	The organism/species - here all data stem from mice
age	The age of the mouse (all mice were 8 weeks here)
sex	The sex of the mouse
infection	The infection state of the mouse, i.e. infected with Influenza A or not infected.
strain	The Influenza A strain; C57BL/6 in all cases.
time	The duration of the infection (in days).
tissue	The tissue that was used for the gene expression experiment, i.e. cerebellum or spinal cord.
mouse	The mouse unique identifier.
ENTREZID	The gene ID for the ENTREZ database
product	The gene product
ensembl_gene_id	The ID of the gene from the ENSEMBL database
external_synonym	A name synonym for the gene
chromosome_name	The chromosome name of the gene
gene_biotype	The gene biotype
phenotype_description	The phenotype description of the gene
hsapiens_homolog_associated_gene_name	The human homologous gene

Processing and Analysis

First, load the data:

```
data <- read.csv(file="rna_seq.csv")
```

Question 1) Count the number of unique genes with expression levels greater than the median expression of the dataset. Ensure you exclude any missing values (NA) from the expression data before performing your calculations.

First, let us calculate the median expression level (removing missing values) and subset the dataframe to rows with above-median expression levels:

```
median_expression <- median(data[["expression"]], na.rm = TRUE)
above_median_expression_subset <- subset(data, expression > median_expression)
```

Now, we can count the number of rows with a unique “gene” identifier:

```
unique_genes <- length(unique(above_median_expression_subset[["gene"]]))
print(unique_genes)
```

```
## [1] 362
```

Question 2) Write code to determine the number of unique mice (based on the mouse column) that were infected (infection == "InfluenzaA") and are of the female sex (sex == "Female").

Again, we can subset to the dataset to only rows that match infection == "InfluenzaA" AND sex == "Female":

```
female_infected_subset <- subset(  
  data,  
  infection == "InfluenzaA" & sex == "Female"  
)
```

and then count the number of rows with a unique "mouse" identifier

```
unique_mice <- length(unique(female_infected_subset[["mouse"]]))  
print(unique_mice)
```

```
## [1] 8
```

Question 3) Find the number of common genes that are expressed in both male and female mice. This means identifying genes that have expression values greater than zero for both sexes.

We can first subset the data to only rows with expression values greater than zero, then subset again to split by sex. For each sex-based subset, we will take only the “gene” column:

```
expressed_subset <- subset(data, expression > 0)
male_unique_subset <- unique(
  subset(expressed_subset, sex == "Male")["gene"]
)
female_unique_subset <- unique(
  subset(expressed_subset, sex == "Female")["gene"]
)
```

Now that we have a vector of genes with an expression level greater than zero for each sex, we can intersect the two sets, and find its length, giving us the number of common genes expressed in both male and female mice:

```
common_genes <- length(
  intersect(male_unique_subset, female_unique_subset)
)
print(common_genes)
```

```
## [1] 130
```

Question 4) For each mouse, identify the gene with the lowest expression level on chromosome 1, ensuring that the expression value is greater than zero. The final result should be a dataframe with two columns: mouse and lowest_gene. You might use `rbind()` to add the results iteratively.

First, we will subset the data to rows with an expression level greater than zero, and a gene that is part of chromosome 1:

```
chromosome_one_subset <- subset(data, chromosome_name == 1 & expression > 0)
```

Next, we can split this subset into a list, where each element is a data frame containing information concerning a unique mouse. Then, we can find the element with the lowest expression, and re-combine all list elements into a singular dataframe:

```
list_per_mouse <- split(chromosome_one_subset, chromosome_one_subset[["mouse"]])
lowest_expression_genes <- lapply(list_per_mouse, function(mouse_df) {
  mouse_df[which.min(mouse_df[["expression"]]), c("mouse", "gene")]
})
recombined_df <- do.call(rbind, lowest_expression_genes)
```

Finally, we can rename columns to their expected names:

```
colnames(recombined_df) <- c("mouse", "lowest_gene")
print(recombined_df, row.names = FALSE)
```

```
##  mouse lowest_gene
##    1          Boll
##    2         Lrrc52
##    4          Gpr39
##    5        Gm28209
##    6        Gm11605
##    7        Slc26a9
##    8        Gm28609
##    9        Gm5523
##   10         Cdk15
##   11        Gm15850
##   12        Gm6177
##   13       Serpinb10
##   14         Mir149
##   15        Prss56
##   16        Mif-ps1
##   18        Slc23a3
##   19        Gm16070
##   20        Ptgs2os
##   21        Gm29183
##   22         Pkp1
##   23         Pkp1
##   24        Gm10193
```

Question 5) Write a function that finds the top 3 genes with the highest expression levels for each chromosome in the dataset. You can utilize `list()` data structure and `order()` function.

We can define a function that splits the dataframe into a list of unique chromosomes, orders each list element's dataframe by expression level in decreasing (higher-first) order, takes the first 3 rows, and rejoins the list into a singular dataframe:

```
get_top_3_chromosomes <- function(input_df) {  
  list_per_chromosome <- split(input_df, input_df[["chromosome_name"]])  
  list_per_chromosome_top3 <- lapply(  
    list_per_chromosome, function(chromosome_df) {  
      chromosome_df[  
        order(  
          chromosome_df[["expression"]],  
          decreasing = TRUE  
        ) [1:3],  
        # Keep these columns  
        c("chromosome_name", "gene", "expression")  
      ]  
    }  
  )  
  recombined_chromosome_df <- do.call(rbind, list_per_chromosome_top3)  
  recombined_chromosome_df  
}
```

Now, we can call the function on our input dataframe, and print the result

```
result <- get_top_3_chromosomes(data)  
print(result, row.names = FALSE)
```

##	chromosome_name	gene	expression
##	1	Atp1b1	71866
##	1	Atp1b1	63790
##	1	Atp1b1	52975
##	10	Dot1l	8520
##	10	Dot1l	6480
##	10	Marcks	6404
##	11	Cnp	14666
##	11	Inpp5j	6577
##	11	Xpo1	6317
##	12	Susd6	5542
##	12	Susd6	5477
##	12	Susd6	4894
##	13	Tppp	12140
##	13	Trim23	5280
##	13	Hmgcs1	4737
##	14	Tsc22d1	9747
##	14	Hnrnpc	9393
##	14	Zfp219	7437
##	15	Desi1	7623
##	15	Plxnb2	6240
##	15	Cyp2d22	2171
##	16	Glyr1	7418
##	16	Snn	5794

##	16	Ppp1r2	4982
##	17	Grm4	13203
##	17	Kat2b	6302
##	17	Slc25a27	4870
##	18	Aqp4	16974
##	18	Impact	6706
##	18	Epc1	3222
##	19	Fads1	7153
##	19	Stx3	3484
##	19	Dpf2	3434
##	2	Bcas1	7352
##	2	Tyro3	4599
##	2	Bcas1	3918
##	3	Dclk1	19016
##	3	Setd7	16765
##	3	Ugt8a	4422
##	4	Eps15	13899
##	4	Eps15	12363
##	4	Ccnl2	9678
##	5	Hopx	6584
##	5	Tmem33	6353
##	5	Vps33a	5258
##	6	Nap115	18287
##	6	Slc6a6	4694
##	6	Rasgef1a	4490
##	7	Brsk2	8578
##	7	Brsk2	7911
##	7	Trim3	7566
##	8	Mt1	18946
##	8	Mt1	6392
##	8	Mau2	4999
##	9	Usp19	6596
##	9	Cryab	3336
##	9	Ptpn23	2963
##	X	Plp1	84540
##	X	Ddx3x	25909
##	X	Ddx3x	20544
##	Y	Ddx3y	5416
##	Y	Uty	1732
##	Y	Gm4017	24

Question 6) To examine the variations in gene expression between males and females, create a Violin plot illustrating the distribution of gene expression levels, separating the data by both **sex** and **infection** status. The y-axis should be on a logarithmic scale (log10). Provide meaningful labels for the axes and ensure the plot is colored by **infection** status.

First, let us import the **ggplot2** and **nord** libraries, and interpret **sex** and **infection** as categorical factors

```
library(ggplot2); library(nord)
sex_factor <- as.factor(data$sex)
infection_factor <- as.factor(data$infection)
```

Now, let us plot the data, displaying sex on the x-axis, gene expression on the y-axis (on a logarithmic scale), and infection as colour:

```
time_factor <- as.factor(data$time)
infection_factor <- as.factor(data$infection)
sex_factor <- as.factor(data$sex)
ggplot(
  data,
  aes(x = time_factor, y = expression, fill = sex_factor, theme = "dark")
) +
  theme_bw() +
  labs(
    title = "Gene expression levels based on sex and infection time",
    y = "Expression (" ~ log[10] ~ "scale)",
    x = "Infection time (days)",
    fill = "Sex"
  ) +
  geom_violin(scale = "count") +
  scale_y_continuous(transform = "log10") +
  scale_fill_nord("algoma_forest")
```


